🔍 Search Investigation

Making Search Flow Stateless: A Comprehensive Analysis

📑 Table of Contents

📋 Overview & Current State

🔑Approach 1: Full Encryption

🎯 Approach 2: Token-Based Caching

⚡ Approach 3: Event-Driven Architecture

🌐 Approach 4: API Gateway-Level Caching

📊 Approach Comparison

✅ Conclusion & Recommendations

📋 Overview & Current State

Current Problem

- The current search flow maintains active sessions during the entire supplier interaction process, creating scalability and reliability challenges.
- The main problem is when a user selects a flight (Fare Confirm) , there is a session which is opened between Server-Side (Backend) and Client-Side (Frontend) , so the booking cycle from the search operation depend on this session
- When user searches for flights with more than one supplier, he/she **MUST** wait for all suppliers' responses which means more waiting time. As there are suppliers respond fastly and others respond slowly

Current Architecture Issues

- **Session Dependency:** Server must remember search parameters and intermediate results
- **State Management:** Each request depends on session state stored in memory
- **Resource Inefficiency:** Duplicate searches with identical criteria trigger separate supplier requests
- **Scalability Concerns:** Session state limits horizontal scaling capabilities
- **Fault Tolerance:** Session loss results in incomplete search processes

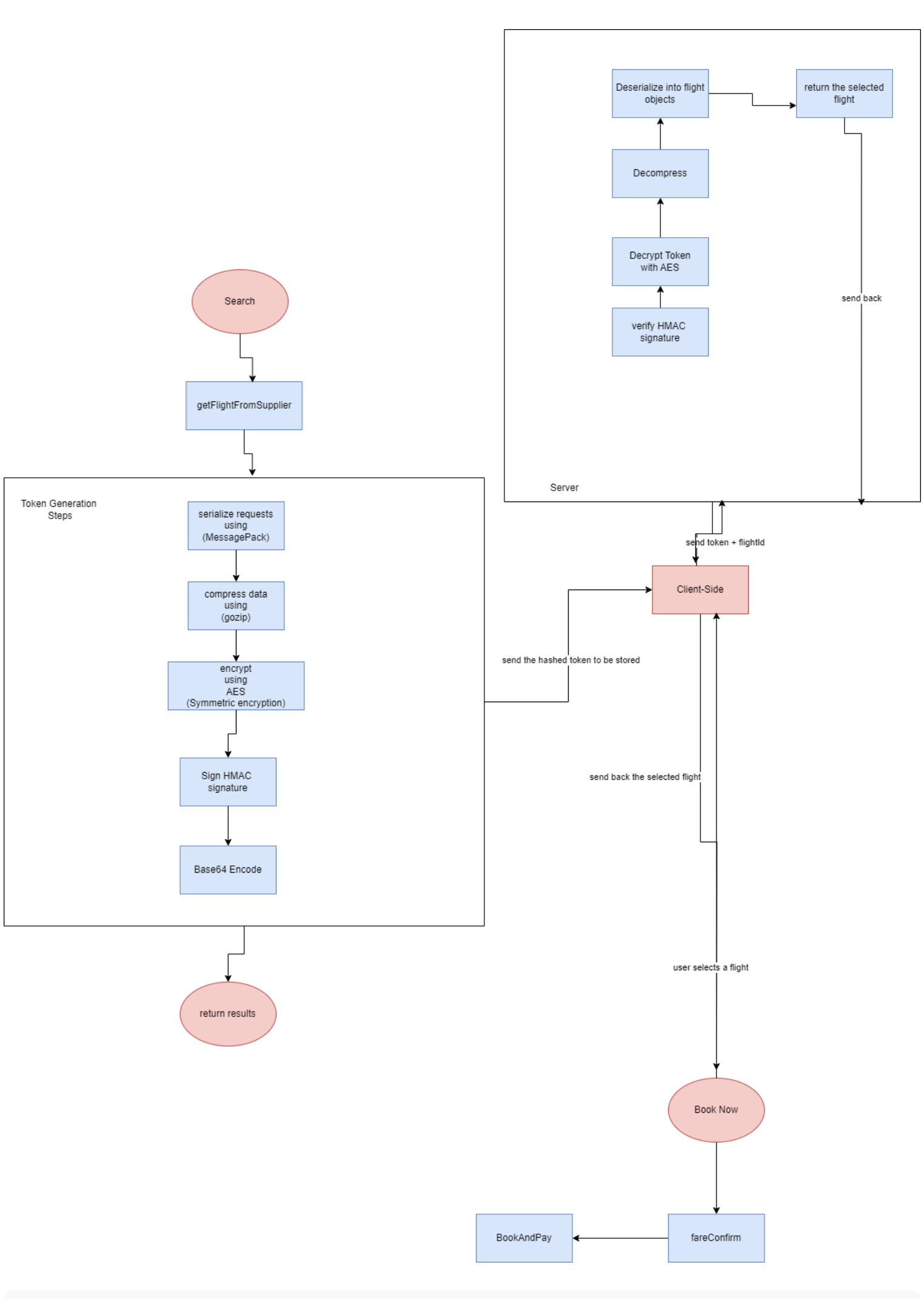
Goal
Transform the search flow into a stateless architecture that improves performance, scalability, and reliability while maintaining user experience quality.

🔑 Approach 1: Full Encryption

Overview

- Instead of storing flight search result on the server **Cache**, the server encodes everything inside a token and sends it back to the client
- The client-side will hold this hashed token which contains the results and the flight request

Workflow



1. **Request Processing:** User submits flight search parameters
2. **Server:** Calls Suppliers to get results and Apply Rule Engine (applyForFareRule)
3. **Encrypt Results :** Token will be processed by differnet steps **(Serialize, Compress, Encrypt, Sign (HMAC), and Base64 Encode)**
4. **Client-Side:** Client stores the generated token
5. **Fare-Confirm:**
   - 5.1 Client-Side will send the encrypt results it holds + selected flightId to the server
   - 5.2 Server:
     - 5.1 Verify HMAC signature
     - 5.2 Decrypt token (encrypted) with AES algorithm
     - 5.3 Decompress back to original flight data
     - 5.4 Deserialize into flight objects.

✅ Advantages

- Lower infrastructue Cost (There is no cache)
- Secured
- Server does not need to look up for flights in cache

❌ Considerations

- Every request will call suppliers , there is no shared results between users with the same search request
- More Complex to generate the token

Technical Implementation

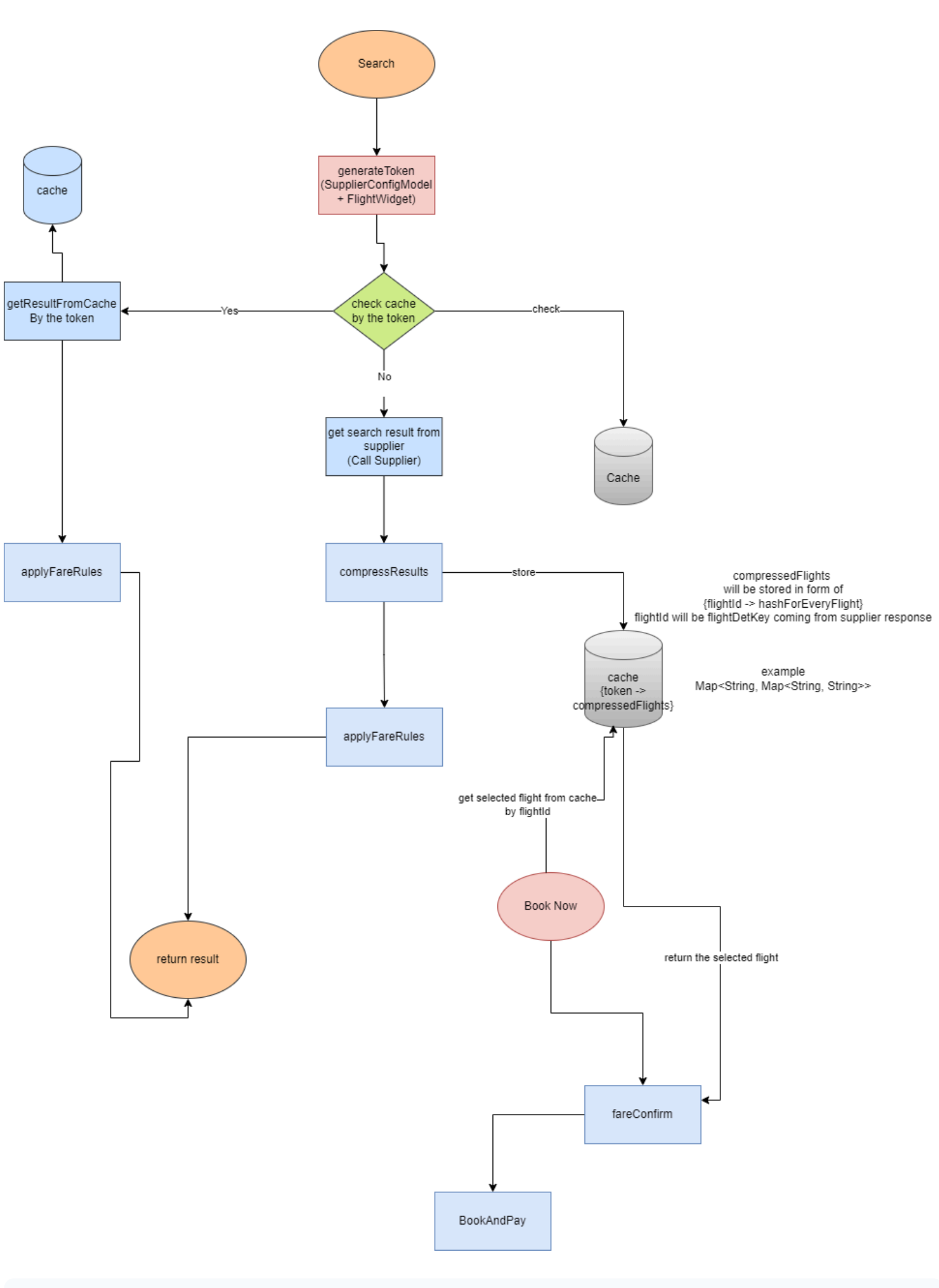
Serialize -> To convert the results into JSON , so they can be compressed
Compress -> To reduce the size of the result
Encrypt (AES alogrithm) -> To ensure that the user cannot modify flights while **Client-Side** holds them
Signature (HMAC algorithm) -> will be changed if any of the data in the results have been changed
Base64 -> To prevent the encrypted results to be changed across HTTP requests

🎯 Approach 2: Token-Based Caching

Overview

This approach implements a stateless design using deterministic tokens and intelligent caching to eliminate session dependencies while optimizing performance.

Workflow



Search
generateToken (SupplierConfigModel + FlightWidget)
cache
getResultFromCache By the token
Yes
check cache by the token
check
No
get search result from supplier (Call Supplier)
Cache
applyFareRules
compressResults
store
compressedFlights will be stored in form of (flightId -> hashForEveryFlight) flightId will be flightDetKey coming from supplier response
cache (token -> compressedFlights)
example Map<String, Map<String, String>>
applyFareRules
get selected flight from cache by flightId
Book Now
return result
return the selected flight
fareConfirm
BookAndPay

1. **Request Processing:** User submits flight search parameters
2. **Token Generation:** Create deterministic JWT token based on supplier credentials and search criteria
3. **Cache Lookup:** Check if results exist in cache using the generated token. Then the result will go through the applyFareRules to apply (Markup, Discount, etc) **(Rule Engine)** for this user
4. **Supplier Interaction:** If cache miss, query suppliers.
5. **Result Processing:** Compress the results using **JWT with compressed payload** OR **Binary Encoding (Message Pack)**
6. **Cache Storage:** Store these processed results in form of {Token -> compressed flights}. In addition to the compressed flights itself will be stored in form of {flightId -> flight}
7. **Rule Engine:** After storing the result in the cache, then apply fare rules
8. **Response Delivery:** Return final results to user

Technical Implementation

Token Generation Strategy
Deterministic JWT: Ensures identical search parameters always generate the same token, enabling effective caching and deduplication.
Components: Supplier Configuration + Flight Search Criteria
Cache Storage:

- Store the results in the cache (Token -> flights) , so other users from different servers with the same search criteria and suppliers can access the results from cache without talking to suppliers again.
- Flights will be stored in the cache in a compressed way , so it will not take a huge amount in the memory
- Store flights using Flyweight pattern way to use the flights in fare confirm without talking to suppliers.

Caching Strategy Options

- **Option A:** Caffeine (L1) + Redis (L2) - Two-tier caching for optimal performance
- **Option B:** Hazelcast - Distributed in-memory grid for cluster-wide caching

Can we git rid of Hazelcast / Cache cluster?

- Cache will still run in embedded mode (for free) **BUT**, the data cannot be shared across the users.
- So we can still use Hazelcast in embedded mode to hold the flight results to use it in fareConfirm

Flyweight Pattern Intro

- Suppose you have a **Text** class with attributes (Description, Color, Font)
- In a traditional way, if you create 1M objects from Text, you will have 3M attributes stored in the memory
- You know that color and font can be shared with all objects created from Text class
- Flyweight pattern let you create font & color attributes **ONLY** once to be shared across the 1M objects

✅ Advantages

- Eliminates duplicate supplier calls
- True stateless architecture
- Improved response times
- Fault tolerance
- Using Flyweight pattern + Token for each flight will help retrieving a specific flight when user selects a flight to book **(FareConfirm)**

❌ Considerations

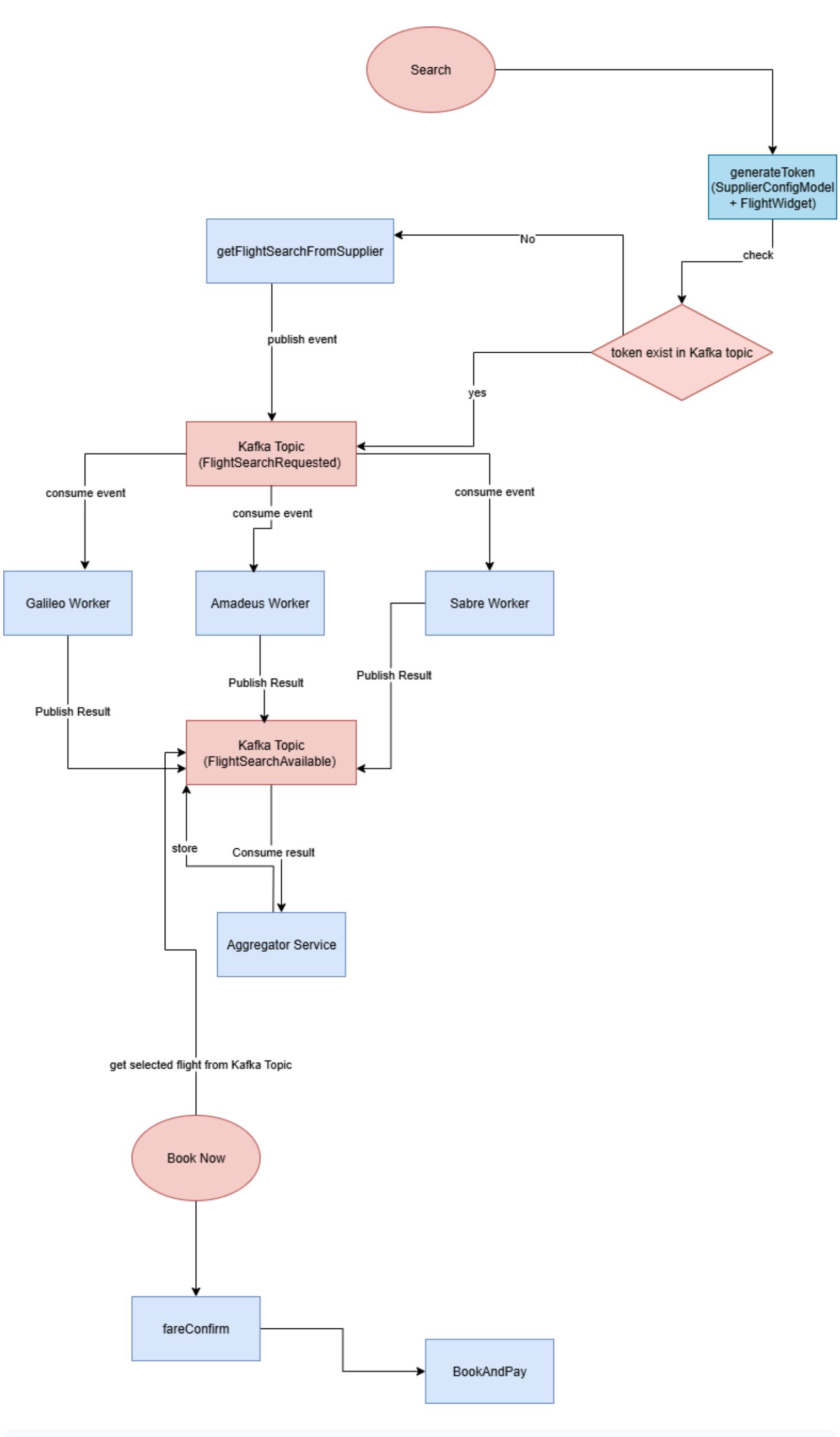
- Cache invalidation complexity
- Memory usage for large result sets
- Token security considerations
- If two users at the same time search with the same criteria using same suppliers, will cause Race conditions in writes

⚡ Approach 3: Event-Driven Architecture

Overview

Instead of synchronous supplier calls holding resources, the service publishes events to Kafka topics with downstream workers processing asynchronously

Workflow



1. **User Search Request:** search parameters (Flight Search Criteria)
2. **Token Generation:** Create **deterministic JWT** token based on supplier credentials and search criteria
3. **Kafka Lookup:** Check if results exist in Result Kafka Topic using the generated token
4. **Event Publication:** If Result Topic miss, Publishes event to Kafka topic instead of direct supplier calls
5. **Supplier Workers (Consumers):** Independent workers process events and fetch supplier results
6. **Results Publishing:**
   - 6.1 Apply Fare Rules after retrieving results from the **Consumers**, ensuring accurate pricing and compliance before confirming the fare to the user.
   - 6.2 Store the processed search results to the **Kafka Topic** with a TTL to ensure they are available for quick retrieval during the **Fare Confirmation** step.
7. **Aggregator Service:** Collect all flights coming from **Consumers (Suppliers)** and return them to the user

✅ Advantages

- Complete decoupling of services
- Excellent scalability
- Fault isolation
- Async processing benefits
- At fareconfirm, Kafka stored the all flights in its **Message Topic** , so when user selects a flight, this flight can be come from Kafka Topic without talk to Client-Side
- We do not need an extra storage for the flights, I can use **Kafka Topic** to temporary store the flights

❌ Considerations

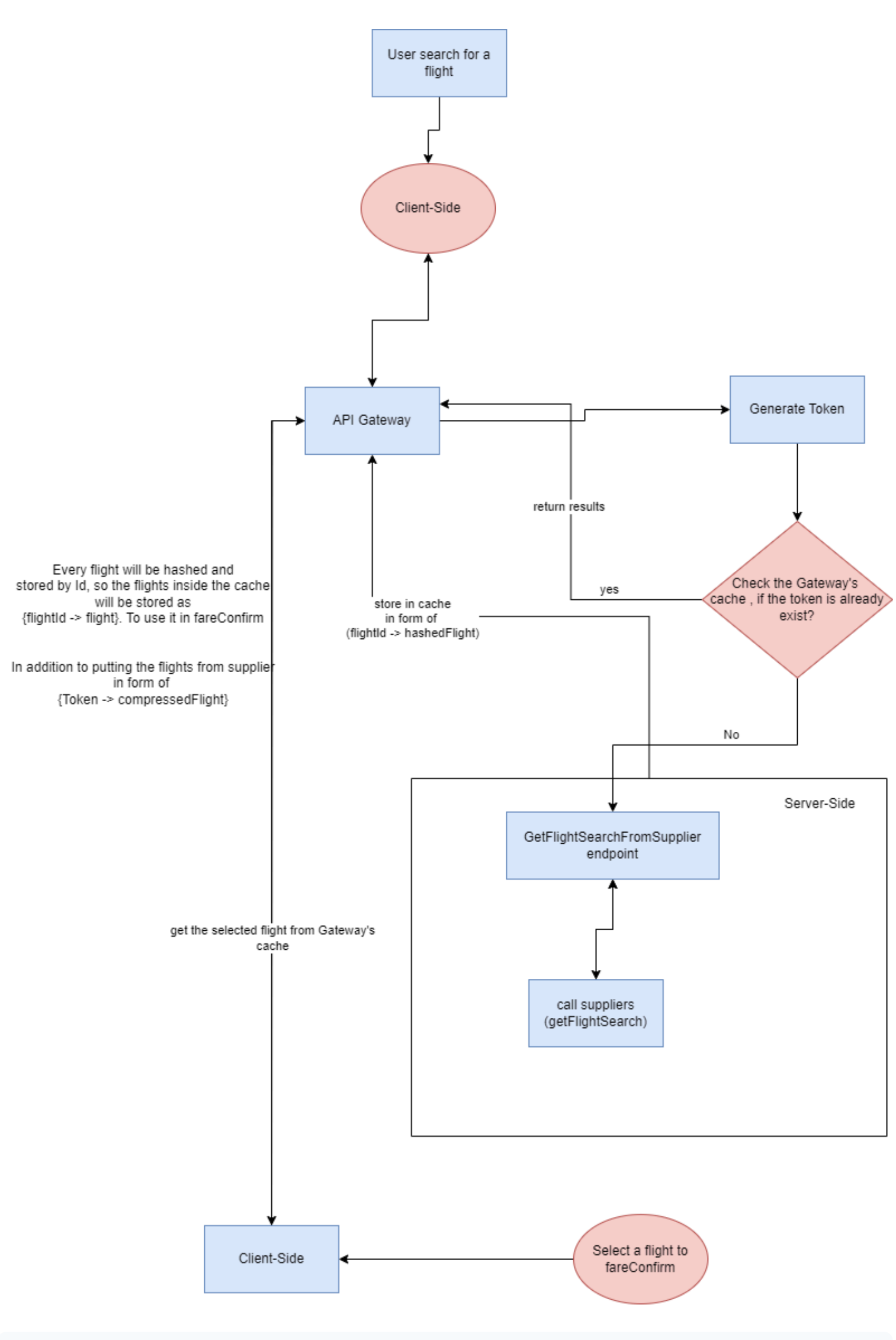
- Increased complexity in implementation
- Event ordering challenges
- Monitoring complexity
- Longer development time

🌐 Approach 4: API Gateway-Level Caching

Overview

Instead of your backend application remembering anything about a user's search (session, in-progress state), you let an API Gateway / Reverse Proxy (like NGINX, AWS API Gateway) handle short-lived caching.

Workflow



User search for a flight
Client-Side
API Gateway
Generate Token
return results
yes
Check the Gateway's cache , if the token is already exist?
Every flight will be hashed and stored by Id, so the flights inside the cache will be stored as (flightId -> flight). To use it in fareConfirm
In addition to putting the flights from supplier in form of (Token -> compressedFlight)
store in cache in form of (flightId -> hashedFlight)
No
Server-Side
GetFlightSearchFromSupplier endpoint
call suppliers (getFlightSearch)
get the selected flight from Gateway's cache
Client-Side
Select a flight to fareConfirm

1. **Search Flow**
   1. **User Search Request:** search parameters (Flight Search Criteria)
   2. **Client-Side:** Send this request to the API Gateway
   3. **API Gateway:** Generate Token , then check whether it exists in the cache and TTL does not expire **(Gateway has a cache inside)**. Then return the result without calling to the server-side
   4. **Server:** If cache miss , then API Gateway will send the request to the server-side. To call suppliers, and applies fare rules. Then return the results to API Gateway to store them in its cache
   5. **Gateway:** Stores responses with TTL in two forms. **{Token -> CompressedFlights}** and **{flightId -> hashedFlight}** for every flight in the response
2. **Fare-Confirm Flow**
   1. **User Select a flight**
   2. **Client-Side:** Send the flightId to the API Gateway
   3. **API Gateway:** Send this **flightId** and **hashedFlight** to the server side
   4. **Server:** Will unHashed this flight then apply fare rules and all operations required to fareConfirm

✅ Advantages

- Very fast
- Cache sits at gateway not on the server, so the server does not be hit everytime
- Simple Setup
- Keeps Client-Side stateless

❌ Considerations

- Still not purely stateless as you will still depend on the existing approach

📊 Approach Comparison

Criteria	Token-Based Caching	Encryption	Event-Driven	API Gateway
Implementation Complexity	Medium	Medium	High	Vey Simple
Performance (Search)	Excellent	Good	Good	Excellent
Fare Confirm Performance	Strong – Token points to cached itineraries, easy lookup of selected flight	Medium -> You need to decrypt -> Decompress -> Deserialize	Strong – Flights stored as events with IDs, easy retrieval for confirm	Weak – Must fallback to backend parsing
Scalability	Very Good	Excellent -> No shared states	Excellent – Kafka partitions handle huge throughput; highly parallel	Good
Development Time	Short	Medium	Long	Very short
Maintenance	Low - Cache management only	Medium	High – Kafka cluster ops	Very Low – Mostly infra-managed
Real-time Capability	Limited – User polls for result	Limited – User polls for result	Excellent – Partial results as they arrive	Limited – Only cache TTL-based, no progressive updates

✅ Conclusion & Recommendations

🎯 Recommended Approach: Kafka (Approach 3)
Rationale: Provides the best of performance improvement, and business value.